Запросы в сеть на корутинах



Retrofit, ошибки, повторы и отмена

Разработка мобильных приложений на Android

Запрос, который не роняет приложение

Что должно стать понятным

- Как Retrofit превращает интерфейс в запросы, а JSON — в data class
- Почему suspend-функция Retrofit не блокирует главный поток
- Как превратить ошибку запроса в состояние экрана
- Как отменить запрос и когда его стоит повторить

Результат: карточка грузится с сервера, не падает без сети и не делает лишних запросов.

Что уже есть

Часть	Отвечает за
HeroApp	Создаёт один GameRepository на всё приложение
GameRepository	Данные приложения: hp, предметы, правила
ItemKey(id)	Какую карточку открыть: в ключе только id
ItemViewModel	Состояние карточки, создаётся фабрикой с id и репозиторием
viewModelScope	Корутины ViewModel, отменяются в onCleared()

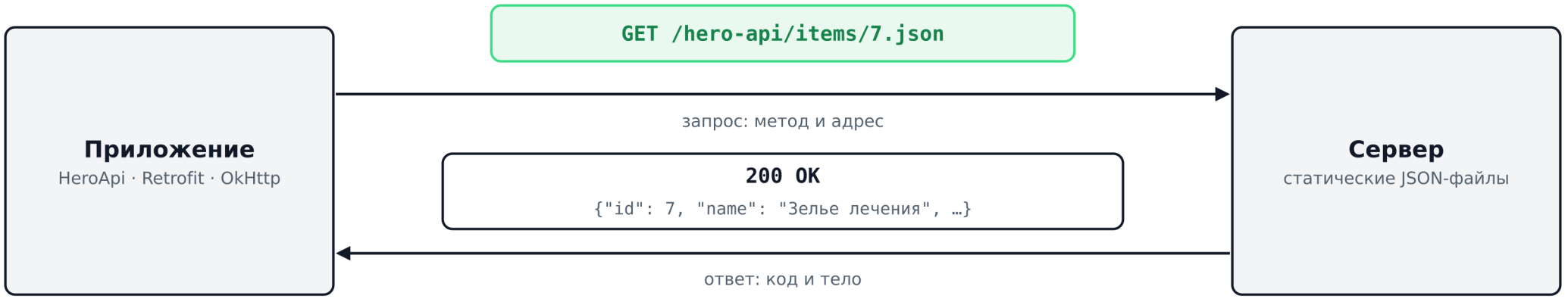
Сегодня в эту схему добавится сервер. Остальное останется на месте.

Описание — на сервере

Открыли зелье, а описание лежит на сервере.

Где сделать запрос? Что показать, пока ждём? А если сети нет?

Запрос и ответ



Три исхода запроса

- **200 → данные**
- **404, 500 → ответ с ошибкой**
- **нет ответа → IOException**

Запрос — это метод и адрес. Ответ — код и тело. Или ответа нет совсем.

Коды ответа

Код	Что значит	Для приложения
200 OK	Данные пришли	Показать
404 Not Found	Такого ресурса нет	«Не найдено», повтор не поможет
500, 503	Ошибка на сервере	Можно повторить позже
Кода нет	Нет сети, тайм-аут, сервер недоступен	Можно повторить

По коду ответа видно, поможет ли повтор.

JSON → data class

```
// items/7.json
// {"id": 7, "name": "Зелье лечения",
//  "heal_power": 30, "rarity": "common",
//  "description": "Пахнет мятой"}

@Serializable
data class ItemDto(
    val id: Int,
    val name: String,
    @SerialName("heal_power") val heal: Int,
    val description: String = "",
)
```

Тот же плагин kotlinx.serialization, что для NavKey.

Другое имя в JSON — @SerialName.

Значение по умолчанию — поле может отсутствовать.

DTO повторяет ответ сервера. Экран получит свою модель.

Retrofit

- Запросы описываем интерфейсом с аннотациями
- Реализацию интерфейса Retrofit создаёт сам
- Функции интерфейса могут быть suspend
- Под капотом — HTTP-клиент OkHttp

Мы пишем, что запросить. Retrofit решает, как это сделать.

Что подключить

Артефакт или настройка	Зачем
com.squareup.retrofit2:retrofit	Интерфейс → HTTP-запросы
com.squareup.retrofit2:converter-kotlinx-serialization	JSON ↔ data class
org.jetbrains.kotlinx:kotlinx-serialization-json	Формат JSON для kotlinx.serialization
com.squareup.okhttp3:logging-interceptor	Запросы в Logcat, по желанию
uses-permission INTERNET в манифесте	Без него — SecurityException

Версии — в libs.versions.toml, разрешение — в AndroidManifest.xml.

Интерфейс API

```
interface HeroApi {

    @GET("items.json")
    suspend fun items(): List<ItemDto>

    @GET("items/{id}.json")
    suspend fun item(@Path("id") id: Int): ItemDto
}
```

@GET — метод и путь от адреса сервера.

{id} заменяется значением параметра @Path.

suspend — ждём ответ, не занимая поток.

Интерфейс описывает запросы. Реализацию напишет Retrofit.

Собираем Retrofit в HeroApp

```
class HeroApp : Application() {
    private val json = Json { ignoreUnknownKeys = true }
    private val converter = json.asConverterFactory(
        "application/json".toMediaType()
    )
    private val api = Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addConverterFactory(converter)
        .build()
        .create(HeroApi::class.java)

    val repository = GameRepository(api)
}
```

ignoreUnknownKeys: лишнее поле в JSON — не ошибка.

BASE_URL должен заканчиваться на «/».

Retrofit создаём один раз, как и репозиторий.

HeroApp создаёт API и передаёт его в репозиторий. Экраны о Retrofit не знают.

Репозиторий: suspend-функция

```
class GameRepository(private val api: HeroApi) {
    // hp, items, useItem — как в прошлой лекции

    suspend fun itemDetails(id: Int): ItemDetails =
        api.item(id).toDetails()
}

data class ItemDetails(
    val id: Int, val name: String,
    val heal: Int, val description: String,
)

private fun ItemDto.toDetails() =
    ItemDetails(id, name, heal, description)
```

suspend: функция ждёт сеть.

Наружу — модель приложения, а не DTO.

Вызывать можно из главного потока.

Репозиторий прячет сеть. ViewModel видит обычную suspend-функцию.

Как Retrofit делает suspend из колбэка

```
// Был колбэк: loadUser(id, onError, onResult)
suspend fun loadUser(id: Int): User =
    suspendCancellableCoroutine { cont ->
        val request = loadUser(
            id,
            onError = { cont.resumeWithException(it) },
            onResult = { cont.resume(it) },
        )
        cont.invokeOnCancellation { request.cancel() }
    }
```

Так Retrofit оборачивает колбэк OkHttp.

Запрос идёт на потоках OkHttp, корутина ждёт без потока.

Отмена корутины отменяет запрос.

suspend-функция Retrofit — это колбэк OkHttp, обёрнутый в корутину.

Нужен ли withContext(Dispatchers.IO)

Что вызываем	Нужен ли withContext
suspend-функция Retrofit	Нет: запрос идёт на потоках OkHttp
delay, withTimeout и другие suspend-функции библиотек	Нет
Call.execute() — блокирующий вызов	Да, Dispatchers.IO
Чтение файла через java.io	Да, Dispatchers.IO
Тяжёлые вычисления над ответом	Да, Dispatchers.Default

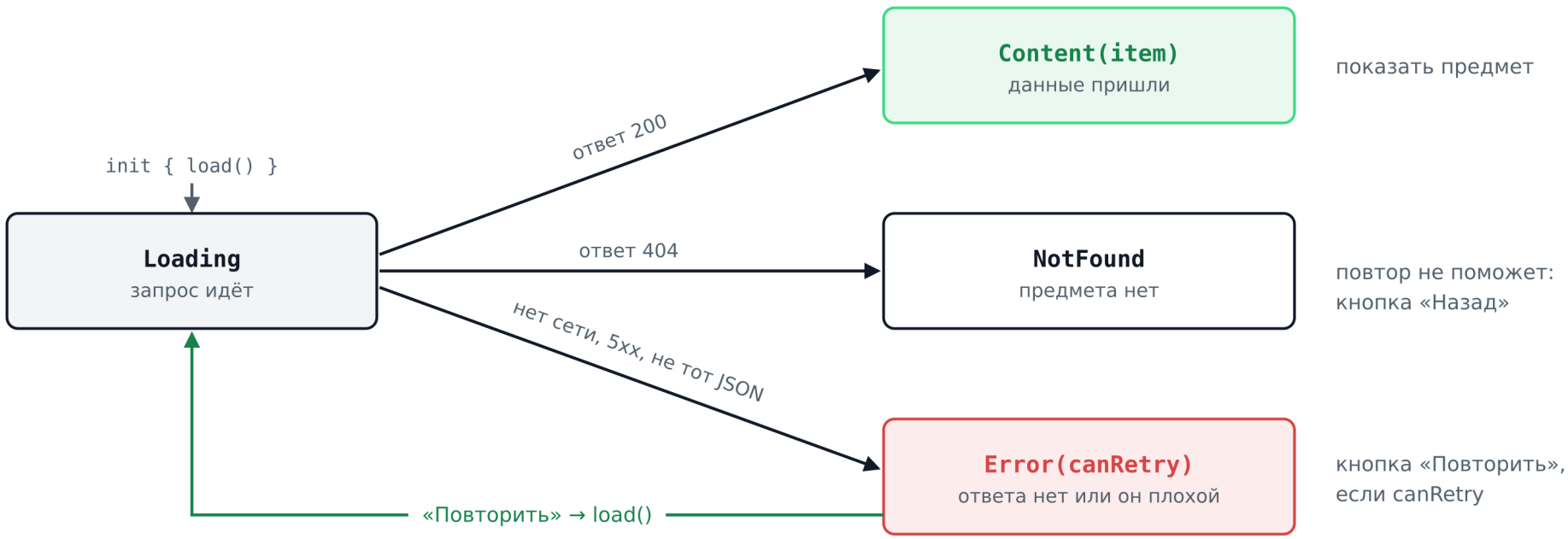
suspend-функция должна быть main-safe. Кто блокирует поток, тот и уходит с него.

Ответа ещё нет

Открыли зелье 7.
Ответ идёт 2 секунды.

Что всё это время на экране? А если сервер ответил «такого предмета нет»?

Состояния карточки



Экран всегда в одном из четырёх состояний. Переходы между ними — события.

ItemUiState: sealed interface

```
sealed interface ItemUiState {
    data object Loading : ItemUiState
    data class Content(val item: ItemDetails) : ItemUiState
    data object NotFound : ItemUiState
    data class Error(val canRetry: Boolean) : ItemUiState
}
```

sealed — других вариантов нет, и компилятор это знает.

У Content и Error есть данные, у остальных нет.

«Загрузка» и «ошибка» сразу — невозможны.

Одно поле состояния вместо набора флагов isLoading, error, item.

ViewModel загружает сама

```
class ItemViewModel(
    private val itemId: Int,
    private val repo: GameRepository,
) : ViewModel() {
    var uiState: ItemUiState by mutableStateOf(Loading)
        private set
    init { load() }
    fun load() {
        viewModelScope.launch {
            uiState = Loading
            uiState = Content(repo.itemDetails(itemId))
        }
    }
}
```

Загрузка начинается при создании ViewModel — один раз на экран.

Корутину запускает ViewModel, а не экран.

Пока без обработки ошибок.

ViewModel владеет корутиной. Экран только читает uiState.

Экран: when по состоянию

```
@Composable
fun ItemScreen(
    state: ItemUiState,
    onRetry: () -> Unit,
    onUse: () -> Unit,
    onBack: () -> Unit,
) {
    when (state) {
        Loading -> LoadingBox()
        is Content -> Details(state.item, onUse, onBack)
        NotFound -> NotFound(onBack)
        is Error -> ErrorBox(state.canRetry, onRetry)
    }
}
```

when без else: забыли состояние — ошибка компиляции.

В ветке is Content state уже Content.

NotFound — экран из прошлой лекции.

Каждому состоянию — свой composable. Экран сам ничего не решает.

Где живут данные

Репозиторий

Данные нужны нескольким экранам.

hp и список предметов.

Живут, пока жив процесс.

Состояние экрана

Ответ нужен одному экрану.

Описание на карточке.

Живёт, пока ключ экрана в стеке.

Общее — в репозитории, разовое — в состоянии экрана.

Включили режим полёта

Включили режим полёта и открыли зелье.

Что будет с приложением? А если сервер ответил 500? А если в JSON нет поля name?

Что может пойти не так

Что случилось	Исключение	Что показать
Нет сети, тайм-аут	IOException	Error, «Повторить»
Ответ 404	HttpException, code() = 404	NotFound
Ответ 500, 503	HttpException, code() = 5xx	Error, «Повторить»
JSON не тот	SerializationException	Error без повтора
Ушли с экрана	CancellationException	Ничего: не ошибка

У каждого исключения — свой ответ пользователю. Отмена — не ошибка.

Ошибка → состояние

```
fun load() {
    viewModelScope.launch {
        uiState = Loading
        uiState = try {
            Content(repo.itemDetails(itemId))
        } catch (e: IOException) {
            Error(canRetry = true)
        } catch (e: HttpException) {
            if (e.code() == 404) NotFound
            else Error(canRetry = e.code() >= 500)
        } catch (e: SerializationException) {
            Error(canRetry = false)
        }
    }
}
```

try — выражение: его значение и есть новое состояние.

Ловим конкретные типы.

Отмену не ловим — она проходит насквозь.

Ошибка запроса становится состоянием экрана, а не падением.

Отмена — тоже исключение

```
// Обновлять описание каждые 5 секунд
viewModelScope.launch {
    while (true) {
        try {
            uiState = Content(repo.itemDetails(itemId))
            delay(5_000)
        } catch (e: Exception) {   // ловит и отмену!
            Log.w("Item", "не удалось обновить", e)
        }
    }
}
```

Ушли с экрана — корутину отменили.

Каждая suspend-функция бросает исключение отмены, а catch его съедает.

Цикл крутится дальше, лог забит.

Отмена в корутинах — исключение. Поймали его — отменили отмену.

Что ломает отмену

Как ловим	Отмена проходит?
catch (e: IOException)	Да: ловим только нужное
catch (e: Exception)	Нет: CancellationException — тоже Exception
runCatching { … }	Нет: ловит всё, включая отмену
catch (e: CancellationException) { throw e } первым	Да: отмену перебрасываем

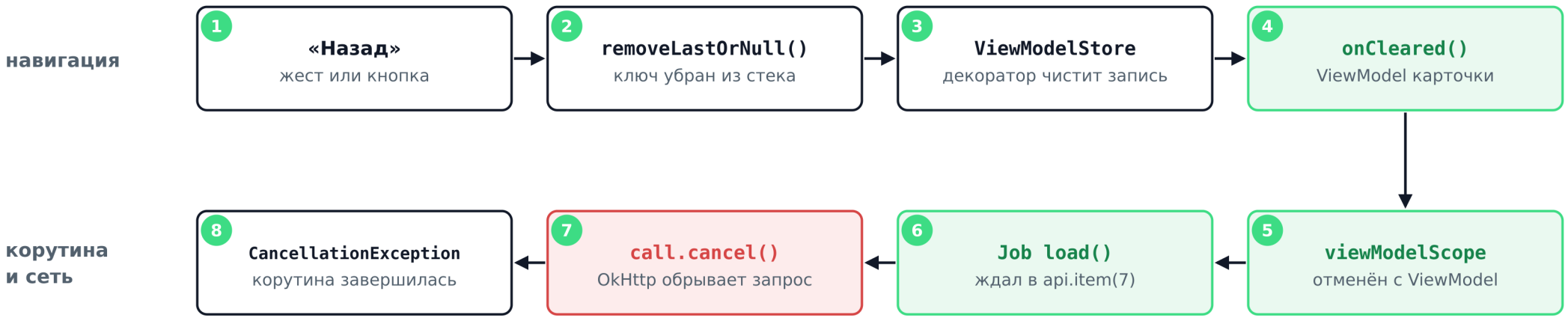
Ловим конкретные исключения. Если нужно поймать всё — отмену перебрасываем.

Пользователь передумал

Открыли зелье и сразу нажали «назад».

Запрос ещё идёт? Кто его остановит? А если во время загрузки повернуть экран?

Цепочка отмены



Для отмены запроса мы не написали ни строки

«Назад» отменяет запрос без единой нашей строчки.

Сколько живёт запрос

Что произошло	Запрос	Что увидит пользователь
Повернули экран	Идёт дальше, второго нет	Ту же загрузку, потом данные
Нажали «назад»	Отменён	Список предметов
Свернули приложение	Идёт дальше	Данные, когда вернётся
Система завершила процесс	Пропал вместе с процессом	Карточку и новую загрузку

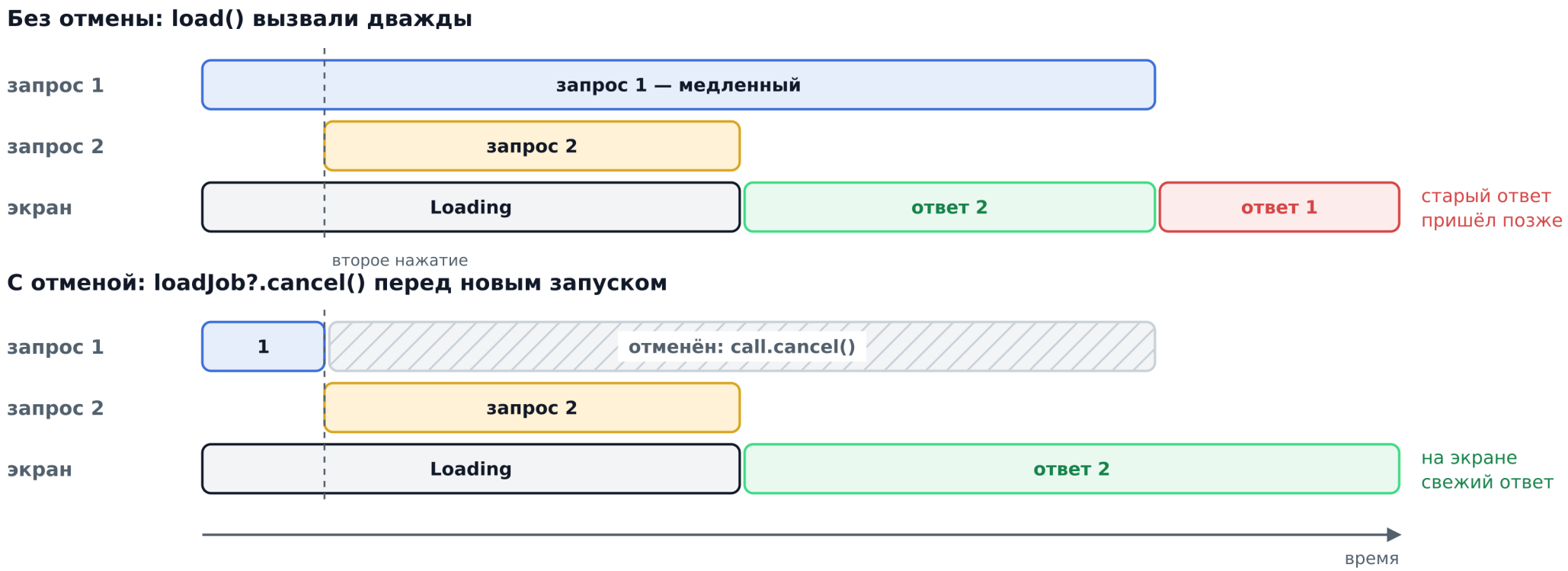
Запрос живёт столько же, сколько ViewModel экрана.

Два нажатия

Сеть вернулась.
Нажали «Повторить» два раза.

Сколько запросов уйдёт? Какой ответ окажется на экране?

Гонка без потоков



Побеждает ответ, который пришёл последним, а не запрос, отправленный последним.

Job: отменить прошлую загрузку

```
private var loadJob: Job? = null

fun load() {
    loadJob?.cancel()
    loadJob = viewModelScope.launch {
        uiState = Loading
        uiState = fetchState()
    }
}

private suspend fun fetchState(): ItemUiState = try {
    Content(repo.itemDetails(itemId))
} catch (e: IOException) {
    Error(canRetry = true)
} // остальные catch — как раньше
```

launch возвращает Job — ручку запущенной корутины.

cancel() отменяет прошлую загрузку вместе с её запросом.

На экране — ответ на последний запрос.

Новый запрос отменяет старый: на экран попадает только свежий ответ.

Отменить старое или пропустить новое

Стратегия	Код	Когда
Отменить старое	loadJob?.cancel(), затем launch	Новый запрос важнее: поиск, фильтр
Пропустить новое	if (loadJob?.isActive == true) return	Запрос тот же: «Повторить», «Обновить»
Выключить кнопку	enabled = state !is Loading	Подсказка в дополнение к одному из двух

Решите заранее, что делать со вторым нажатием. Два одновременных запроса — почти всегда ошибка.

Отмена кооперативная

- Корутина узнаёт об отмене в точке приостановки
- delay, withContext и запросы Retrofit отменяются сразу
- Долгий цикл без приостановки проверяет ensureActive()
- finally выполняется и при отмене

Отмена — просьба. Корутина выполняет её в ближайшей точке приостановки.

Тайм-ауты и лог запросов

```
private val client = OkHttpClient.Builder()
    .callTimeout(15, TimeUnit.SECONDS)
    .addInterceptor(HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BASIC
    })
    .build()

private val api = Retrofit.Builder()
    .baseUrl(BASE_URL)
    .client(client)
    .addConverterFactory(converter)
    .build()
    .create(HeroApi::class.java)
```

У OkHttp по умолчанию тайм-ауты соединения и чтения — по 10 с.

callTimeout ограничивает весь запрос.

Тайм-аут — это IOException.

Тайм-ауты задаём на клиенте. Лог запросов — только в отладочной сборке.

Ловушка withTimeout

```
// Плохо: тайм-аут не попадёт в catch
viewModelScope.launch {
    uiState = try {
        withTimeout(5_000) { fetchContent() }
    } catch (e: IOException) {
        Error(canRetry = true)
    }
}

// Лучше: null вместо исключения
val content = withTimeoutOrNull(5_000) { fetchContent() }
uiState = content ?: Error(canRetry = true)
```

По тайм-ауту withTimeout бросает исключение отмены.

launch считает его обычной отменой.

Экран навсегда остаётся в Loading.

Тайм-аут через withTimeout — тоже отмена. Нужен результат — withTimeoutOrNull.

Поезд въехал в тоннель

Поезд въехал в тоннель.
Запрос не прошёл.

Через секунду сеть вернётся. Повторить сразу? Сколько раз? А если сервер ответил 404?

Что повторять

Что случилось	Повторять?	Почему
IOException: нет сети, тайм-аут	Да, с паузой	Сбой может пройти сам
HTTP 500, 503	Да, с паузой	Сервер может восстановиться
HTTP 404 и другие 4xx	Нет	Ответ не изменится
SerializationException	Нет	Ошибка в программе или на сервере
CancellationException	Никогда	Это не ошибка, а отмена

Повторяем временные сбои и только идемпотентные запросы.

Повтор с растущей паузой

```
suspend fun <T> retryIO(
    times: Int = 3,
    initialDelay: Long = 500,
    block: suspend () -> T,
): T {
    var pause = initialDelay
    repeat(times - 1) {
        try { return block() } catch (e: IOException) { }
        delay(pause)
        pause *= 2
    }
    return block()   // последняя: ошибка уйдёт наверх
}
```

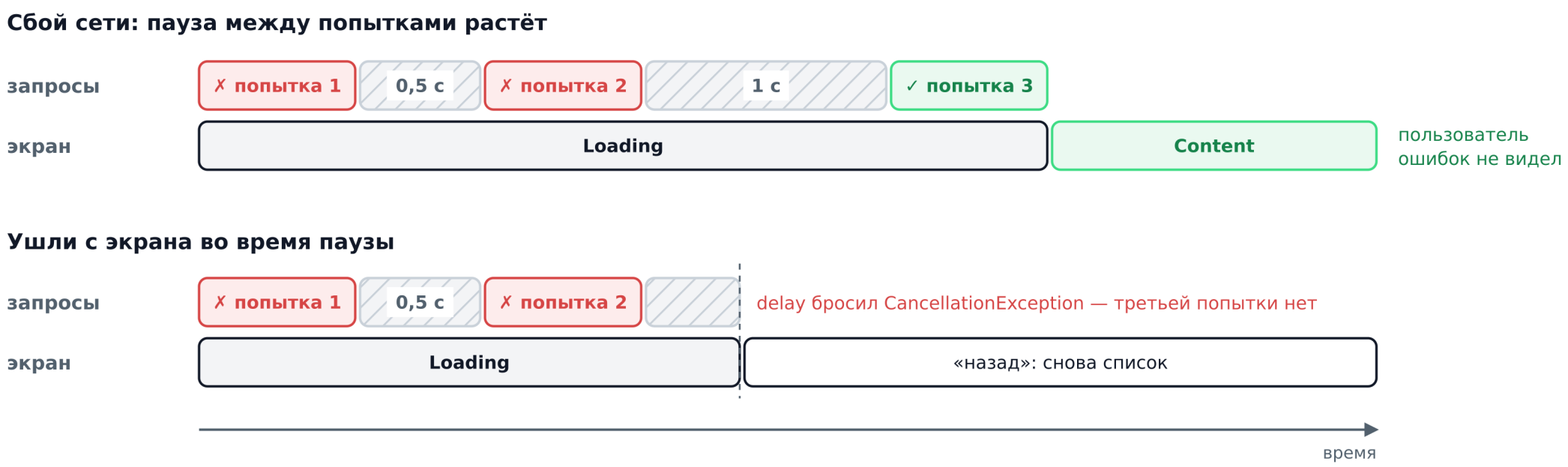
Пауза растёт: 0,5 с, 1 с, 2 с…

delay отменяемый: ушли с экрана — повторы прекратились.

Ловим только IOException.

Повтор — обычный цикл с delay. Отмена работает сама.

Повторы во времени



Пользователь видит одну загрузку, а не три ошибки.

Автоматически и по кнопке

```
// GameRepository: тихие повторы при сбоях
suspend fun itemDetails(id: Int): ItemDetails =
    retryIO { api.item(id) }.toDetails()

// ItemViewModel: повтор по кнопке
fun onRetry() = load()

// ItemRoute
ItemScreen(
    state = viewModel.uiState,
    onRetry = viewModel::onRetry,
    // onUse, onBack — как раньше
)
```

Повторы при сбое — правило данных, они в репозитории.

Кнопка — решение пользователя.

Пока идут тихие повторы, на экране Loading.

Сначала несколько тихих попыток, потом решает пользователь.

Кто за что отвечает

Часть	Отвечает за
HeroApi	Какие запросы есть у сервера
HeroApp	Создаёт Retrofit, API и репозиторий — один раз
GameRepository	Данные, запросы и повторы при сбоях
ItemViewModel	Корутина загрузки, Job, ошибка → состояние
ItemUiState	Что видит пользователь: четыре варианта
ItemScreen	Показывает состояние, сообщает о событиях

Сеть знает репозиторий. Корутиной владеет ViewModel. Экран только показывает.

Частые ошибки: настройка

Симптом	Что проверить
SecurityException: missing INTERNET permission	Строку uses-permission в манифесте
IllegalArgumentException: baseUrl must end in /	«/» в конце BASE_URL
CLEARTEXT communication … not permitted	Адрес http://, а нужен https://
SerializationException: unknown key	ignoreUnknownKeys = true
MissingFieldException	Поле есть в DTO, но нет в JSON
NetworkOnMainThreadException	Блокирующий execute() на главном потоке

Эти ошибки видны при первом же запуске: читаем текст исключения.

Частые ошибки: корутины

Симптом	Что проверить
Падает в режиме полёта	Нет try/catch вокруг запроса
Падает на ответе 500	Ловится IOException, а HttpException — нет
Вечная загрузка	withTimeout без обработки
После «назад» в логе новые запросы	catch (e: Exception) или runCatching в цикле
На экране старый ответ	Не отменили прошлый Job
Два запроса после поворота	Загрузка в LaunchedEffect, а не во ViewModel

Спрашиваем: кто владелец корутины и куда делось исключение?

Проверяем понимание

- Почему suspend-функцию Retrofit можно вызвать из главного потока?
- Чем HttpException отличается от IOException?
- Почему catch (e: Exception) ломает отмену?
- Что будет с запросом при повороте и при «назад»?
- Зачем хранить Job загрузки?
- Какие ошибки стоит повторять, а какие нет?

Ответ должен объяснять поведение программы, а не только название API.

Смотрим вживую

- Режим полёта: экран ошибки и «Повторить»
- Медленная сеть и «назад»: Canceled в логе
- Предмета нет на сервере и битый JSON
- catch (e: Exception) в цикле: лог после «назад»



Каждую демонстрацию сначала предсказываем, потом запускаем.

Практика: инвентарь с сервера

- Загружать список предметов из items.json в репозиторий
- Экран списка: Loading, Error с «Повторить», Content
- Тихие повторы при сбое сети, без повторов на 404
- Второе нажатие «Обновить» не запускает второй запрос

Проверить: режим полёта, поворот и «назад» во время загрузки.

Что дальше

- Хранение на устройстве: Room и DataStore
- Flow и StateFlow: данные, которые меняются сами
- Внедрение зависимостей библиотекой
- Картинки из сети: Coil

Разовый запрос — suspend-функция. Поток изменений — Flow.

Материалы для повторения

- Retrofit: документация и список изменений
- Android: Connect to the network, Best practices for coroutines
- Kotlin: Cancellation and timeouts, Exception handling
- Google: codelab «Get data from the internet»

Ссылки на соответствующие темы есть в заметках к слайдам.